CT2 Question Paper - Question 4

1. Introduction

The complexity of modern embedded systems makes developing hardware and software in sequence highly inefficient and risky. To accelerate time-to-market and ensure correctness, engineers rely heavily on Modeling and Simulation. Simulating a system before physical hardware is available allows for early verification of algorithms and logic. However, simulations must be tailored to specific needs, differentiating between high-level functional verification and low-level architectural accurate models. Complementing this, the Unified Modeling Language (UML) provides behavioral modeling tools—specifically Use-Case and Activity diagrams—that allow developers to map out the dynamic flow of the system and user interactions long before a single line of code is simulated or executed.

2. Core Technical Explanation

a) Compare Simulation: High-Level vs. Low-Level

Simulation is the process of imitating the behavior of a real-world process or system over time using a software model. The fidelity of this model determines whether it is high-level or low-level.

High-Level Simulation (Functional/System-Level):

- **Objective:** To verify the algorithmic logic, control laws, and overall system functionality independent of the target hardware architecture.
- **Working Principle:** It abstracts away the hardware specifics (registers, bus timing, pipeline stalls). The software is compiled for and executed on the host PC.
- **Tools:** MATLAB/Simulink, LabVIEW, Python models.
- **Application:** Prototyping a PID control algorithm for a drone. You simulate the mathematics of the controller against a mathematical model of the drone's physics to ensure stability, without caring whether the final code will run on an ARM Cortex-M4 or an ESP32.

Low-Level Simulation (Instruction Set/Cycle-Accurate):

- **Objective:** To verify how the specific cross-compiled binary will execute on the exact target hardware architecture, focusing on timing, memory access, and processor constraints.
- **Working Principle:** It utilizes an Instruction Set Simulator (ISS). The ISS interprets the cross-compiled machine code (hex file) step-by-step, mimicking the exact behavior of the

target CPU's ALU, registers, and memory map. Cycle-accurate simulators even model bus latencies and cache hits.

- **Tools:** QEMU, Keil μVision Simulator, specialized SoC emulators.
- **Application:** Verifying that an interrupt service routine (ISR) executes within a strict 50-microsecond deadline, taking into account the specific clock speed and instruction cycle counts of the target microcontroller.

b) Explain in Detail the UML Behavioral Modelling: Use-Case, Activity Diagrams

While Structural UML diagrams (Class/Component) describe what the system *is*, Behavioral diagrams describe what the system *does*.

Use-Case Diagrams:

- **Purpose:** To capture the functional requirements of the system from the perspective of external entities. It models the system's boundary and how it interacts with the outside world.
- **Core Elements:**
  - **Actors:** External entities (human users, other systems, or sensors) that interact with the system. Represented as stick figures.
  - **Use Cases:** Specific functionalities or goals the system provides to the actors. Represented as ovals.
  - **System Boundary:** A box enclosing the use cases, separating the system from the external actors.
  - **Relationships:** Associations between actors and use cases; `<<include>>` (mandatory behavior) and `<<extend>>` (optional behavior) between use cases.

Activity Diagrams:

- **Purpose:** To model the control flow and data flow from one activity to another within a system. It is essentially an advanced, object-oriented flowchart that supports parallel processing and concurrency.
- **Core Elements:**
  - **Action/Activity Nodes:** Represents a step or process. Represented as rounded rectangles.
  - **Control Flow:** Arrows indicating the sequence of execution.
  - **Initial/Final Nodes:** Solid circle for start, circled solid circle for the end.
  - **Decision/Merge Nodes:** Diamond shapes for branching logic (if/else).

- **Fork/Join Nodes:** Heavy solid bars. A *fork* splits one flow into multiple concurrent parallel flows. A *join* waits for multiple concurrent flows to finish before proceeding. This is critical for modeling multi-threaded RTOS behavior.

3. Mathematical / Theoretical Support

Simulation fidelity directly impacts timing analysis. In low-level Cycle-Accurate Simulation, execution time (T_{exec}) is mathematically verified based on instruction cycles:

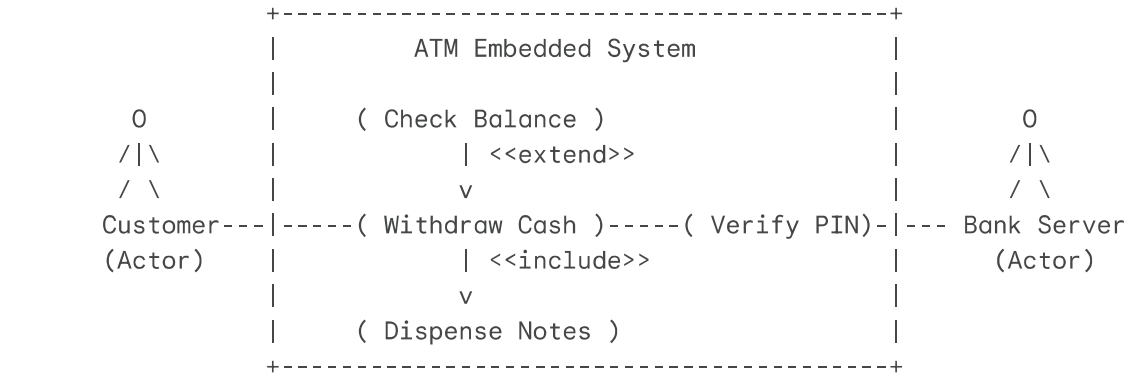
$$T_{exec} = \sum_{i=1}^{N} (C_i \times T_{clk}) + T_{stall}$$

Where:

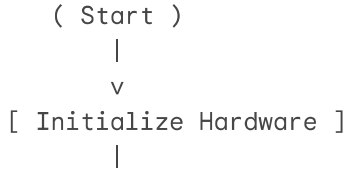
- N is the number of instructions executed.
- C_i is the number of clock cycles required for instruction i.
- T_{clk} is the period of the processor clock ($1/f$).
- T_{stall} accounts for pipeline stalls or memory wait states. High-level simulation completely ignores this equation, assuming instantaneous execution of logic, which is why low-level simulation is required for hard real-time verification.

4. Relatable Diagram: UML Behavioral Diagrams

Use-Case Diagram (Automated ATM):



Activity Diagram (Concurrent RTOS Data Sampling):



```
       ====== FORK BAR ====== (Splits into parallel tasks)
       |                     |
       v                     v
 [ Sample ADC ]       [ Read I2C Sensor ]
       |                     |
       v                     v
 [ Filter Data ]      [ Calibrate Data ]
       |                     |
       ====== JOIN BAR ====== (Waits for both to finish)
                 |
                 v
      [ Package Network Frame ]
                 |
            (  End  )
```

5. Industrial Case Study / Real-World Example

Flight Control System Development for a Commercial Drone:

- **UML Behavioral Modeling:** Before coding, engineers create a **Use-Case diagram** identifying the Pilot (Actor) interacting with functions like `Take-off`, `Auto-Return`, and `Manual Flight`. An **Activity diagram** is mapped for the `Auto-Return` use case, using Fork/Join nodes to show that the GPS navigation thread and the Obstacle Avoidance thread must run concurrently to guide the drone home.
- **Simulation Phase:**
  - First, **High-Level Simulation** (MATLAB/Simulink) is used to simulate the drone's aerodynamics and tune the PID flight controllers in a virtual 3D environment to ensure it flies stably in simulated wind.
  - Once the C-code is generated, **Low-Level Simulation** (QEMU) is used to execute the compiled ARM binary. This verifies that the software stack does not cause a memory overflow and that the critical motor-control interrupt always completes within its 1-millisecond deadline, ensuring real-time safety constraints are met before risking physical hardware.

6. Advantages and Limitations

Comparison: High-Level vs. Low-Level Simulation

Feature	High-Level Simulation	Low-Level Simulation (ISS)
Focus	Algorithm logic, mathematics, system dynamics.	Architecture timing, registers, binary execution.

Speed	Very fast execution on host PC.	Much slower; cycle-accurate emulation is computationally heavy.
Hardware Dependency	Agnostic; requires no knowledge of target CPU.	Highly dependent; requires specific CPU/SoC models.

Comparison: Use-Case vs. Activity Diagrams

Feature	Use-Case Diagram	Activity Diagram
Perspective	External view; interaction between users and system.	Internal view; flow of control and data between processes.
Best Used For	Gathering requirements and defining system scope.	Detailing complex algorithms and concurrent/parallel RTOS logic.

7. Conclusion

Effective embedded system engineering bridges the gap between conceptual requirements and silicon reality. UML behavioral modeling, via Use-Case and Activity diagrams, provides the necessary framework to translate user requirements into structured, concurrent logical flows. Simulation then acts as the ultimate verification engine. By combining the rapid, algorithmic